

# MuJoCo 기반 탱크 강화학습 시뮬레이션

개인 프로젝트 진행 정리 — 국소 내비게이션(로컬 조향/속도 제어) 정책 학습

## 1. 프로젝트 개요

팀 프로젝트로 진행 중인 Unity 기반 탱크 시뮬레이션(자율주행 트랙, D\* Lite 경로탐색 + YOLO 객체 탐지 + Flask 서버 통신 구조)과는 별개로, 강화학습(RL)을 직접 적용해보는 개인 프로젝트로 시작함. 팀 프로젝트는 이미 RL 없이 완성되어 있어, 팀 시나리오에 억지로 맞추지 않고 RL 고유의 강점(실시간 라이다 기반 회피)이 살아나는 형태로 문제를 독립적으로 재정의함.

Unity 시뮬레이션은 배포판 바이너리 형태로만 제공되어 내부 소스에 접근할 수 없고, 정해진 REST API로만 통신 가능한 구조였음. 이 때문에 Unity 안에서 직접 강화학습 루프(자동 리셋, 대량의 반복 스텝, 병렬 환경)를 자동화하는 것이 사실상 불가능하다고 판단, 별도의 물리 시뮬레이터에서 정책을 학습한 뒤 결과만 이식하는 방식을 택함.

### 1.1 시뮬레이터 선택

Unity ML - Agents(소스 접근 필요로 배제), 경량 커스텀 Python 시뮬레이터, MuJoCo, NVIDIA Isaac Lab(GPU 의존성 높아 배제) 등을 비교한 결과, 무료 · 오픈소스이며 CPU만으로도 충분히 빠르고 Gymnasium/Stable - Baselines3와 바로 연동되는 MuJoCo를 채택함. 학원/개인 PC(Intel i5 - 13400, 16GB RAM, 내장그래픽) 사양으로도 충분히 감당 가능한 것으로 확인.

## 2. 문제 정의 및 단계별 로드맵

RL이 담당할 범위를 "전체 자율주행 대체"가 아니라 "기존 PID 컨트롤러가 담당하던 로컬 조향/속도 제어"로 한정. 상위 목적지 계산(D\* Lite)은 그대로 두고, 그 경로를 실제로 따라가는 매 순간의 조향 판단만 학습된 정책으로 대체하는 구조를 목표로 설계함. 이후 개인 프로젝트로 완전히 독립시키면서, 최종적으로는 사전 지도 정보 없이 라이다만으로 내비게이션하는 문제로 재정의함.

| 단계  | 목표                                                     | 상태                        |
|-----|--------------------------------------------------------|---------------------------|
| A   | 정적 장애물 + 라이다만으로 목적지 찾기 (D* Lite 의존성 제거, 장애물/시작/목표 랜덤화) | 진행 중 (환경 구축 완료, 1차 학습 실행) |
| 2단계 | 실제 Unity 맵 데이터를 반영한 복잡한 배치에서 검증                        | 예정                        |
| B   | 동적 장애물(움직이는 다른 탱크 등) 회피 추가                             | 예정                        |
| C   | 부분 지도 정보 + 동적 장애물 통합 (최종 목표)                           | 예정                        |

## 3. 시뮬레이션 환경 구성

### 3.1 좌표계 및 스케일

- Unity(x, y=높이, z)와 MuJoCo(x, y, z=높이) 좌표계 차이를 명시적으로 매핑 (Unity z  $\leftrightarrow$  MuJoCo y).

- 실제 Tank Challenge API 문서 기준 탱크 크기(폭 3.667 / 높이 1.582 / 길이 8.066)와 맵 크기(300×300)를 그대로 반영.
- 장애물 좌표는 실제 API의 x\_min/x\_max/z\_min/z\_max 스키마를 그대로 사용해, 추후 실제 맵 데이터(Unity .map 파일)를 그대로 파싱해 재사용 가능하도록 설계.

### 3.2 탱크 물리 모델

- 차동구동(differential drive) 방식: 좌우 바퀴 2개 + 전방 캐스터 1개 (3점 지지 구조).
- 초기에는 좌우 바퀴 2개 + 앞뒤 캐스터 2개(4점 지지)로 구성했으나, 직진 시 좌우로 번갈아 미끄러지는 현상(부정적 구조에 의한 수치적 불안정)이 발생해 캐스터 1개로 축소, 이후 완전히 해결됨.
- 액추에이터는 순수 토크 제어(motor)에서 목표 각속도를 직접 지정하는 velocity 제어로 전환하여, 실제 시뮬레이터의 최고 속도 제한(40km/h)을 물리적으로 재현.
- velocity 액추에이터의 게인(kv)이 탱크 질량(약 4톤급) 대비 지나치게 낮아 목표 속도에 크게 못 미치는 문제를 겪었으며, kv를 대폭 상향(3000)하여 해결.

### 3.3 장애물 및 맵 생성

- 매 에피소드(reset)마다 시작 위치, 목표 위치, 장애물 배치를 모두 무작위로 재생성하여 특정 배치를 암기하는 과적합을 방지.
- 격자(grid) 기반 배치 방식을 채택: 맵을 일정 간격의 격자로 나누고, 각 칸에 확률적으로 장애물(정사각형 또는 긴 벽 형태)을 배치. 시작점 · 목표점 주변은 안전 반경을 두어 장애물이 생기지 않도록 함.
- 초기에는 시작~목표를 잇는 직선 경로 주변에만 장애물을 배치하는 방식을 시도했으나, 장애물이 한 지점에 뭉치거나 개수가 불안정하게 적어지는 문제가 있어 격자 기반 방식으로 전환, 맵 전체를 고르게 활용하도록 개선.

### 3.4 조명 및 시각화

기본 점광원(point light)이 원점 근처만 밝히고 넓은 맵 외곽은 어두워지는 문제를 방향광(directional light) + 헤드라이트(ambient)로 전환하여 해결.

## 4. 강화학습 설계

### 4.1 알고리즘: SAC (Soft Actor-Critic)

조향각 · 속도처럼 연속적인 값을 출력해야 하는 문제 특성상, 이산 행동에만 적용 가능한 DQN 계열은 배제하고 액터-크리틱 계열을 채택. 그중 SAC는 엔트로피 보너스를 통해 탐험을 유지하며 학습 안정성이 높고, replay buffer 기반으로 샘플 효율이 좋아 짧은 개발 기간 안에 결과를 확인하기에 적합하다고 판단하여 선정. 구현은 Stable-Baselines3 라이브러리의 SAC("MlpPolicy", ...)를 사용.

### 4.2 관측 공간 (13차원)

| 구성 요소   | 차원 | 설명                                                                              |
|---------|----|---------------------------------------------------------------------------------|
| 라이다 거리  | 9  | 전방 부채꼴(약 140도)로 배치한 rangefinder 센서, 정규화된 거리값. 실제 API의 lidarPoints 구조를 단순화하여 모사. |
| 속도      | 1  | 탱크의 현재 이동 속도(정규화)                                                               |
| 목표까지 거리 | 1  | 목표 지점까지 남은 거리(정규화)                                                              |
| 목표 방향   | 2  | 탱크 진행 방향 기준 목표의 상대 각도(sin, cos)                                                 |

### 4.3 액션 공간 (2차원, 연속)

정책은  $[v, w]$  (각각  $-1 \sim 1$ )를 출력.  $v$ 는 전진/후진 성분,  $w$ 는 회전 성분이며, 학습 환경에서는 이를 좌우 바퀴 목표 각속도로 변환하여 차동구동 운동학에 따라 물리적으로 반영. 실제 Tank Challenge API는 "방향(command) + 가중치(weight,  $0.1 \sim 1.0$ )" 형태의 명령 구조를 쓰므로, 배포 시에는 동일한  $v, w$  값을 그대로 받아 API 포맷(moveWS/moveAD)으로 변환하는 별도의 어댑터 함수를 준비해 두어, 정책 자체는 학습 · 배포 환경에 무관하게 재사용 가능하도록 설계함.

### 4.4 보상 함수

```
reward = -0.01 × (■■■■ ■■) [■ ■■]
```

- 충돌 시 -20.0의 추가 감점
- 목표 도달(거리 3m 이내) 시 +50.0의 추가 보상
- 목표에 가까워질수록 매 스텝 감점이 줄어드는 형태로, 정책이 지속적으로 목표 방향으로 이동하도록 유도. 향후 장애물 회피 비중을 높이거나, 급격한 조향에 페널티를 주는 형태로 개선 여지가 있음.

## 5. 개발 과정에서 발견하고 해결한 주요 이슈

| 이슈                      | 원인                                      | 해결                                           |
|-------------------------|-----------------------------------------|----------------------------------------------|
| 탱크 몸체가 프로펠러처럼 회전하며 날아감  | 몸체와 바퀴 geom이 겹쳐 자기 자신과 충돌 계산됨           | 부모-자식 body 간 충돌을 <contact> <exclude>로 명시적 제외 |
| 직진 시 좌우로 번갈아 미끄러짐       | 바퀴 2개+캐스터 2개(4점 지지)의 부정정 구조로 인한 수치적 불안정 | 캐스터를 1개로 줄여 3점 지지(삼각대) 구조로 변경                |
| 목표 속도 대비 실제 속도가 크게 못 미침 | velocity 액추에이터 게인(kv)이 탱크 질량 대비 지나치게 낮음 | kv를 50 → 3000으로 대폭 상향                        |

| 이슈                      | 원인                                                                         | 해결                                           |
|-------------------------|----------------------------------------------------------------------------|----------------------------------------------|
| 학습 시 보상이 전혀 개선되지 않음(초기) | 고정된 장애물 · 목표 배치로 인해 탱크가 거의 움직이지 않는 상태로 전 스텝이 시간초과로 종료                      | 환경 검증용 디버그 스크립트로 원인 진단 후 물리 파라미터 재조정         |
| 에피소드가 항상 시간초과로 종료       | 물리 timestep 세분화(0.0005) 이후 실제 시뮬레이션 시간 예산이 에피소드당 5초로 지나치게 짧아짐(목표 거리 대비 부족) | max_episode_steps를 시뮬레이션 시간 기준으로 재계산하여 상향 조정 |
| 학습된 정책 확인 중 뷰어 화면이 멈춤   | reset() 시 MuJoCo 모델 · 데이터 객체를 매번 새로 생성하나, 뷰어는 최초 생성 시점의 객체 참조를 그대로 유지      | 에피소드마다 뷰어를 새로 여는 구조로 변경                      |

## 6. 현재까지 진행 상황

격자 기반 랜덤 맵(정사각형 장애물 + 벽 형태 혼합) 환경에서 1차 SAC 학습(약 96,000 스텝, 약 16분) 실행. 이전(고정 장애물 · 고정 목표) 환경에서는 학습 신호가 전혀 나타나지 않았던 것과 달리, 이번에는 에피소드 평균 보상(ep\_rew\_mean)이 -1,380에서 -996까지 꾸준히 개선되는 추세를 확인. 에피소드 평균 길이(ep\_len\_mean)도 후반부에 소폭 감소하기 시작하여, 목표에 조기 도달하는 사례가 발생하기 시작했음을 시사.

다만 학습된 정책을 직접 시각화하는 과정에서, 에피소드당 허용 시간(당시 max\_episode\_steps=2000)이 시뮬레이션 시간 기준 5초에 불과해 목표 거리가 먼 다수의 에피소드가 애초에 시간 내 완주가 불가능했던 구조적 문제를 발견함. 이를 max\_episode\_steps 상향 조정으로 수정하고, 이어서 추가 학습을 진행할 예정.

## 7. 다음 단계

- max\_episode\_steps를 시뮬레이션 시간 예산에 맞게 재조정 후 이어서 학습 진행
- 학습된 정책을 뷰어로 직접 관찰하며 라이다 기반 회피 행동이 실제로 나타나는지 정성적으로 확인
- 2단계: 실제 Unity 맵 데이터(.map 파일)를 파싱하여 더 현실적인 배치에서 검증
- B단계: 움직이는 장애물(동적 회피) 도입
- C단계: 부분 지도 정보 + 동적 장애물을 통합한 최종 목표 환경 구성